EXPLORATORY DATA ANALYSIS

Chapter 22

# EDA for Different Data Domains

Adapting investigative questions to the structure and meaning of real-world data

**Course tools:** Python, NumPy, Pandas, Matplotlib's object-oriented API, Seaborn, and selected domain-aware techniques.

This chapter adapts the EDA mindset to different real-world contexts. The goal is not to memorize a separate set of charts for every industry, but to understand how row meaning, data generation, domain constraints, and consequences shape the investigation.

# Learning objectives

• Identify the observational unit, grain, population, and data-generating process in unfamiliar domains.

• Choose domain-relevant EDA questions, summaries, visualizations, and quality checks.

• Explore business, customer, transaction, education, healthcare-style, web, product, survey, text, geospatial, and operational data.

• Recognize denominator errors, repeated measures, exposure differences, time ambiguity, and selection effects.

• Distinguish event counts from rates, outcomes from processes, and records from people or entities.

• Apply privacy and interpretation safeguards to sensitive or high-stakes data.

• Build a reusable domain-adaptation checklist.

> **Central principle:** the same statistical operation can answer different questions—or mislead—depending on what a row and a variable mean.

# 1. A universal framework for domain-aware EDA

A domain changes the meaning of the data, not the mathematical definitions of mean, median, quantile, or correlation. Before computing anything, identify the entity being observed and the process that generated each record.

| Question | What to establish | Why it matters |
|---|---|---|
| What is one row? | Person, account, order, event, visit, session, device, day? | Prevents confusing records with unique entities |
| What is the grain? | One row per entity, entity-period, or event? | Determines valid counts and joins |
| Who is included? | Target population, eligibility, coverage, exclusions | Defines the scope of conclusions |
| What is measured? | Units, scale, definitions, source, timing | Prevents semantic errors |
| What is the denominator? | People, eligible people, sessions, exposure time, attempts? | Makes rates interpretable |
| What is the time meaning? | Event, transaction, report, ingestion, or update time? | Prevents temporal misinterpretation |
| What constraints apply? | Business rules, logical bounds, domain relationships | Enables meaningful quality checks |
| What could be sensitive? | Personal, financial, educational, or health information | Guides privacy and responsible handling |

A useful first deliverable is a data dictionary with a row-grain statement, variable definitions, units, allowed values, missing-value meanings, and source lineage.

# 2. Review of earlier chapters (Chapters 1–20)

This review excludes Chapter 21, which is reviewed separately below.

• **Ch. 1 — EDA:** investigate iteratively; observations are not explanations.

• **Ch. 2 — Dataset understanding:** define rows, columns, variable types, scales, and grain.

• **Ch. 3 — Data quality:** inspect completeness, validity, consistency, uniqueness, accuracy, and timeliness.

• **Ch. 4 — Numerical variables:** examine location, spread, distribution, and unusual observations.

• **Ch. 5 — Categorical variables:** inspect counts, proportions, rare categories, and denominators.

• **Ch. 6 — Date/time:** validate parsing, time zones, durations, and temporal coverage.

• **Ch. 7 — Descriptive statistics:** choose robust summaries and interpret conventions.

• **Ch. 8 — Distributions:** use frequency, density, ECDF, and shape to understand data behavior.

• **Ch. 9 — Distribution shape:** interpret skewness, kurtosis, and quantiles carefully.

• **Ch. 10 — Distribution visualization:** select appropriate distribution plots and scales.

• **Ch. 11 — Relationships:** investigate form, strength, nonlinearity, and confounding.

• **Ch. 12 — Multivariate visualization:** use facets, heatmaps, and encodings responsibly.

• **Ch. 13 — Association:** choose measures by variable type; association is not causation.

• **Ch. 14 — Covariance and dependence:** distinguish zero correlation from independence.

• **Ch. 15 — Grouped analysis:** make aggregation rules, group sizes, and denominators explicit.

• **Ch. 16 — Outlier detection:** flag unusual observations for investigation, not automatic deletion.

• **Ch. 17 — Unusual observations:** validate source records and compare relevant peers.

• **Ch. 18 — High-dimensional EDA:** organize variables and avoid over-reading large scans.

• **Ch. 19 — Time series:** inspect cadence, gaps, seasonality, breaks, and temporal context.

• **Ch. 20 — Sampling and bias:** understand target population, selection, missingness, and coverage.

# 3. Review of Chapter 21 — Automated EDA and Data Profiling

Chapter 21 covered automated profiling, schema inference, univariate and relationship scans, quality alerts, ydata-profiling, Sweetviz, large-data sampling, reproducibility, and drift comparisons.

> **Expected reasoning for the Chapter 21 assignment/project:** start with a manual baseline, document tool versions and sampling, validate important alerts with targeted analysis, and distinguish automated observations from analyst interpretation. If no assignment was submitted, this is a reasoning guide rather than a grading result.

A profiler can report a negative account balance; only business rules and source context can tell you whether it is invalid, expected, or meaningful.

# 4. Business and customer data

## 4.1 Observational units

Common tables have one row per customer, account, subscription, support interaction, order, or customer-month. These grains are not interchangeable. A customer table and an order table have different row counts and different meanings.

## 4.2 Useful questions

• How many distinct customers, accounts, and active entities are represented?

• How are new, returning, active, inactive, and churned customers defined?

• How do tenure, order frequency, basket value, and support contacts vary?

• Do outcomes differ across acquisition channels, regions, or cohorts?

• Are repeated records legitimate events or duplicate ingestion?

## 4.3 Denominator discipline

A conversion rate may use visitors, eligible visitors, sessions, leads, or unique people. A churn rate may use customers active at the beginning of a period, not all rows in an export. State the denominator and time window.

```
# Customer-level table: one row per customer assumed.
customer_summary = (
    customers.groupby("customer_segment", dropna=False)
    .agg(
        customers=("customer_id", "nunique"),
        mean_tenure=("tenure_months", "mean"),
        median_spend=("total_spend", "median")
    )
)

# Order-level table: count orders and customers separately.
order_summary = (
    orders.groupby("region", dropna=False)
    .agg(
        orders=("order_id", "nunique"),
        customers=("customer_id", "nunique"),
        revenue=("order_amount", "sum")
    )
)
```

For client work, validate whether revenue is gross or net, whether refunds are included, and whether cancelled orders are retained.

# 5. Transaction and financial-style records

A transaction table may contain authorizations, captures, settlements, reversals, refunds, chargebacks, and adjustments. One purchase can create several rows. Summing every amount without understanding event types can double-count money.

• Check transaction identifiers and expected uniqueness at the correct event grain.

• Inspect currency codes, units, decimal precision, and conversion rules.

• Separate purchases, refunds, reversals, and fees.

• Compare event time, settlement time, and ingestion time.

• Check impossible state transitions and negative durations.

• Inspect extreme amounts by transaction type and relevant peer group.

```
# Interpret amounts only after currency and event semantics.
transaction_summary = (
    transactions.groupby(
        ["currency", "event_type"], dropna=False
    )
    .agg(
        records=("transaction_id", "size"),
        distinct_transactions=("transaction_id", "nunique"),
        median_amount=("amount", "median"),
        total_amount=("amount", "sum")
    )
)
```

A high total is not automatically high economic activity. Understand event definitions, duplicate events, currencies, and reversals. Financial records may also be legally or contractually sensitive.

# 6. Education data

Rows may represent students, enrollments, student-course terms, assessments, attendance events, or school-year aggregates. A student can appear many times; repeated rows are not necessarily duplicates.

• Clarify whether attendance is recorded per day, class, or session.

• Check whether grades are comparable across courses, instructors, assessment types, and years.

• Investigate missing later-period records: transfer, dropout, or incomplete capture may differ.

• Compare cohorts and courses without treating group differences as innate ability.

• Recognize assessments nested within students, classes, and schools.

Averages can hide differences in grading scales, course difficulty, and student composition. Define the observational unit before calculating an attendance percentage.

# 7. Healthcare-style and clinical records

This section concerns data structure and responsible EDA, not clinical advice or diagnosis. Health-related data can contain high-stakes measurements, repeated observations, coding systems, and strong privacy constraints.

• Rows may represent patients, encounters, admissions, prescriptions, laboratory results, or observations.

• Repeated measurements within a person may be irregularly spaced.

• Facilities and devices may use different protocols and units.

• Codes may represent diagnoses, procedures, billing categories, or classifications.

• Distinguish not measured, not ordered, refused, unavailable, and not applicable.

• Check whether data includes only people who reached a facility or received a test.

• Investigate changes in coding systems, clinical practice, devices, and facility coverage.

Selection into care or testing can make observed records unrepresentative of the broader population. A dataset of tested patients is not automatically a dataset of all people with a condition. Do not infer prevalence without an appropriate sampling and testing framework.

> **Privacy rule:** avoid identifiable patient details in notebooks, plots, filenames, and exported reports. Suppress or aggregate small groups when disclosure risk exists.

# 8. Web analytics and product analytics

A clickstream table often has one row per event. One person can generate many events, sessions, and devices. Event counts, session counts, and user counts answer different questions.

• Check event names, property definitions, schema versions, and unexpected nulls.

• Inspect event ordering and duplicate delivery.

• Distinguish event time from server receipt time.

• Check sessionization rules and cross-device identity limitations.

• Look for bot traffic, internal testing, consent effects, and tracking blockers.

• Inspect funnel eligibility and the denominator at each step.

```
# Event-level summary: user and event counts differ.
event_summary = (
    events.groupby("event_name", dropna=False)
    .agg(
        event_rows=("event_id", "size"),
        unique_events=("event_id", "nunique"),
        users=("user_id", "nunique"),
        sessions=("session_id", "nunique")
    )
    .sort_values("event_rows", ascending=False)
)
```

A missing event does not always mean the user did not perform the action; instrumentation may have failed or been blocked. Funnel drop-off can reflect tracking differences as well as user behavior.

# 9. Operational logs, IoT, and system telemetry

Operational data may represent requests, errors, jobs, device readings, alerts, or periodic heartbeats. Its quality is closely tied to system architecture and logging policy.

• Identify whether rows represent events, periodic snapshots, or aggregated windows.

• Check timestamp precision, timezone, clock drift, and out-of-order arrival.

• Distinguish no event from a missing heartbeat or logging outage.

• Inspect duplicate delivery, retries, and idempotency keys.

• Compare error rates with request volume or operating time.

• Check sensor calibration, units, device replacement, and firmware changes.

Counts alone can mislead: 100 errors among 1,000 requests differs from 100 errors among 10 million requests. Use exposure-aware rates when denominators are available.

```
service_health = (
    logs.groupby("service", dropna=False)
    .agg(
        requests=("request_id", "nunique"),
        errors=("is_error", "sum")
    )
)
service_health["error_rate"] = (
    service_health["errors"] /
    service_health["requests"].replace(0, np.nan)
)
```

# 10. Survey and questionnaire data

Survey data combines sampling design, question wording, response behavior, and skip logic. Preserve the distinction between different reasons for nonresponse.

• Inspect question wording, response options, ordering, and survey mode.

• Document skip patterns and structural missingness.

• Distinguish refusal, don't know, not applicable, and not reached.

• Check item nonresponse and unit nonresponse separately.

• Review weights, strata, clusters, and design documentation when supplied.

• Compare response rates across meaningful groups and collection waves.

Do not replace special survey codes with ordinary numeric values before interpreting their meaning. A value such as 99 may mean “not stated,” not ninety-nine.

## 11. Text, reviews, and support conversations

Text data is often semi-structured. A row may be a message, conversation, review, ticket, or document. The same author may produce many records, and multiple languages or moderation processes may be involved.

• Check text length, empty strings, encoding problems, and duplicated content.

• Inspect language mix, tokenization artifacts, templates, and boilerplate.

• Separate user-authored text from agent replies and system messages.

• Check timestamps, conversation boundaries, and thread reconstruction.

• Consider spam, moderation, selection, and deletion policies.

• Protect personal information and avoid exposing quoted text unnecessarily.

Sentiment labels or topic categories may reflect annotator rules, language coverage, or platform selection. EDA can inspect label distributions and disagreement; it cannot assume labels are objective ground truth.

## 12. Geospatial and location-linked data

Location data may be points, routes, regions, addresses, or aggregated areas. Coordinate reference systems and spatial aggregation choices matter.

• Validate latitude/longitude ranges and coordinate reference systems.

• Check whether coordinates represent a person, device, store, event, or approximate centroid.

• Inspect spatial coverage and missingness by region.

• Account for boundary effects and different area sizes.

• Use rates or density measures with suitable population/exposure denominators.

• Consider location privacy and re-identification risk.

Raw counts by region are not directly comparable when regions differ in population, area, or observation coverage. Mapping precise individual locations can create serious privacy risks.

## 13. Cross-domain patterns and recurring traps

| Trap | What goes wrong | Better practice |
|---|---|---|
| Wrong row meaning | Rows are called people or customers when they are events | Write the grain explicitly |
| Wrong denominator | Counts compared across unequal exposure | Report eligible population or exposure |
| Repeated measures | Repeated observations treated as independent entities | Separate entity and observation counts |
| Time ambiguity | Event, report, and ingestion times are mixed | Define timestamp semantics |
| Selection bias | Observed records assumed to represent everyone | Describe frame and capture process |
| Unit mismatch | Measurements from different scales are compared | Normalize units with documented rules |
| Pipeline effects | Joins or filters silently alter coverage | Audit keys, row counts, and exclusions |
| Sensitive reporting | Small groups or raw records expose individuals | Aggregate, suppress, and minimize data |
| Overgeneralization | Sample patterns stated as population facts | Match conclusions to sampling evidence |

## 14. Choosing analysis by domain

| Domain | Useful summaries | Useful displays |
|---|---|---|
| Customers | Distinct entities, tenure, spend, retention | Cohort lines, histograms, grouped boxplots |
| Transactions | Counts and amounts by event/currency | Time series, amount distributions, category bars |
| Education | Enrollment, attendance, assessment distributions | Cohort plots, distributions by course/term |
| Healthcare-style | Repeated measures, missingness, encounter patterns | Patient timelines, distributions, missingness maps |
| Web/product | Users, sessions, events, funnel denominators | Funnel charts, event timelines, cohort retention |
| Operations | Error rates, uptime, latency quantiles | Time plots, tail distributions |
| Survey | Response rates, weighted/unweighted summaries | Category bars, subgroup missingness plots |
| Geospatial | Coverage, rates, spatial density | Maps with appropriate aggregation |

# 15. A domain-adaptation template in Python

This pattern is metadata-driven. It does not attempt to guess the meaning of columns.

```
def domain_inventory(
    df, entity_key=None, event_key=None, time_col=None
):
    result = {
        "shape": df.shape,
        "columns": pd.DataFrame({
            "dtype": df.dtypes.astype(str),
            "missing_n": df.isna().sum(),
            "missing_rate": df.isna().mean(),
            "unique_n": df.nunique(dropna=True)
        }),
        "duplicate_rows": int(df.duplicated().sum())
    }

    if entity_key:
        result["distinct_entities"] = int(
            df[entity_key].nunique(dropna=True)
        )

    if event_key:
        result["duplicate_event_keys"] = int(
            df.duplicated(event_key, keep=False).sum()
        )

    if time_col:
        parsed = pd.to_datetime(df[time_col], errors="coerce")
        result["invalid_time_n"] = int(parsed.isna().sum())
        result["time_min"] = parsed.min()
        result["time_max"] = parsed.max()

    return result
```

Add domain-specific checks separately. A transaction audit needs currency and event-type rules; an education audit needs enrollment and assessment semantics; a telemetry audit needs expected cadence and heartbeat rules.

# 16. How to write domain-aware findings

A professional finding connects an observed pattern to its context without claiming more than the data supports.

| Weak statement | Stronger statement |
|---|---|
| “Customers spend more in Region A.” | “Among recorded completed orders in this extract, median order value is higher in Region A; ord |
| “Students with low attendance perform poorly.” | “In this dataset, lower recorded attendance is associated with lower assessment scores; course, |
| “The system was down.” | “The telemetry stream has a 42-minute gap; heartbeat and ingestion logs are needed to distingu |
| “The disease is common.” | “The condition appears in X% of these recorded encounters; this selected clinical dataset canno |

Use four parts: **observation** (what the data shows), **scope** (which records and period), **interpretation** (plausible meaning), and **limitation** (what remains unknown).

# 17. Common mistakes

• Applying a generic EDA template without understanding row grain.

• Calling every row a customer, patient, student, or user.

• Comparing raw counts without exposure or eligibility denominators.

• Ignoring repeated measurements and nested data structures.

• Assuming all timestamps refer to the same event.

• Treating missing events as proof that nothing happened.

• Comparing amounts or measurements without checking units and definitions.

• Interpreting domain-specific codes without a codebook.

• Overlooking privacy, consent, and disclosure risks.

• Making population or causal claims from selected observational records.

## 18. Summary

• The domain determines what rows, variables, events, and denominators mean.

• Start with the observational unit, grain, target population, and data-generating process.

• Business, transaction, education, healthcare-style, web, operational, survey, text, and geospatial data have distinct structures and pitfalls.

• Repeated observations, exposure, temporal semantics, and collection coverage matter across domains.

• Domain rules turn generic statistical checks into meaningful quality checks.

• Privacy and high-stakes interpretation are part of professional EDA.

• Communicate observations with explicit scope, interpretation, and limitations.

## 19. Assignment

**Part A — Domain selection**

Choose one domain: business/customer, transaction, education, healthcare-style, web/product, operational/IoT, survey, text, or geospatial data.

**Part B — Data model**

• Describe the target population and observational unit.

• State the row grain and identify entity keys, event keys, and time fields.

• Define at least eight variables, including units and allowed values where relevant.

• Describe the data-generation and collection process.

**Part C — Domain-specific EDA plan**

• Write at least ten questions that matter in the selected domain.

• Identify appropriate summaries, plots, and denominators for each question.

• List at least five domain-specific validity or consistency rules.

• Identify likely missingness, selection, and measurement problems.

**Part D — Implementation**

• Run a general data-quality and profiling audit.

• Add at least five domain-aware checks.

• Produce at least three visualizations chosen for the domain.

• Compare at least two meaningful groups or time periods, where appropriate.

• Document privacy or disclosure safeguards.

**Part E — Findings**

Write three findings using observation, scope, interpretation, and limitation. Include one tempting conclusion that the dataset does not justify.

## 20. Coding project — Domain-Aware EDA Case Study

Extend your cumulative EDA master notebook. Build one complete case study using a dataset from a domain you can explain. A public, synthetic, or properly de-identified dataset is suitable.

• Create a domain overview and data dictionary.

• Document target population, row grain, entities, events, timestamps, and denominators.

• Run a reusable general EDA audit.

• Implement at least five domain-specific checks with clear reasons.

• Investigate missingness and coverage in the context of the collection process.

• Produce distribution, relationship, grouped, and temporal or spatial views as appropriate.

• Investigate unusual records rather than automatically deleting them.

• Identify at least three limitations or potential biases.

• Write a concise client-style report with findings, evidence, and recommendations.

• Keep the analysis reproducible and protect sensitive information.

**Acceptance criteria:** domain meaning is understood before statistics are computed; row grain and denominators are explicit; checks are justified by the data-generating process; visualizations answer stated questions; and conclusions stay within the evidence.

**Suggested deliverables:** cumulative notebook, data dictionary, domain-check table, selected figures, and a concise EDA report.

## 21. Preview of Next Chapter

**Chapter 23 — Designing a Reproducible EDA Workflow**

We will turn course techniques into a repeatable professional workflow: project structure, data intake, configuration, validation, modular analysis, versioning, reproducible outputs, and audit trails. The goal is to make an EDA project understandable, rerunnable, and maintainable—not merely a notebook that worked once.

**Boundary:** we will focus on reproducible analytical practice and workflow design, not production data engineering or model deployment.